

Perfect 👍 . Let's **expand Module 1 (SELECT)** into a **full lecture-style class note** with deeper explanations, different use cases, and step-by-step practice.



SQL CLASS NOTE – MODULE 1: SELECT

1. Introduction

The **SELECT statement** is the **foundation of SQL**.

It allows you to **ask questions** (queries) from your database and **retrieve specific information**.

- Without **SELECT** , a database is like a locked library: it has books (data), but you can't read them.
- With **SELECT** , you're asking:

“Show me this book, this page, or just these sentences.”

2. Syntax

```
SELECT column1, column2, ...  
FROM table_name;
```

- **SELECT** → tells the database *what you want to see*.
- **FROM** → tells the database *where (which table)* to get it from.
- ***** → means “all columns”.

3. Practical Examples with `Schoo1DB`

Example 1: Select all student data

```
SELECT *  
FROM Students;
```

- This retrieves **all columns** (ID, Name, Age, Gender, GradeLevel).
- ⚠ Use carefully — if a table has thousands of columns/rows, this can be slow.

Example 2: Select specific columns

```
SELECT FirstName, LastName, Age
FROM Students;
```

- This retrieves only **FirstName, LastName, and Age**.
- Useful when you don't need everything.

Example 3: Select data from another table

```
SELECT CourseName, Teacher
FROM Courses;
```

- Displays **all courses** and their **teachers**.

Example 4: Select with aliases (renaming)

```
SELECT FirstName AS "First Name", LastName AS "Last Name"
FROM Students;
```

- AS renames columns in the result set.
- Great for making reports more **readable**.

Example 5: Select with calculations

```
SELECT FirstName, Age, Age + 1 AS "Age Next Year"
FROM Students;
```

- You can perform **arithmetic operations** on numeric fields.
- Here we calculate how old each student will be next year.

Example 6: Select distinct values

```
SELECT DISTINCT GradeLevel  
FROM Students;
```

- Removes duplicates, showing each grade **only once**.

Example 7: Select with string concatenation

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName  
FROM Students;
```

- Combines `FirstName` + `LastName` into one column **FullName**.

Example 8: Select with ordering (preview for later module)

```
SELECT FirstName, Age  
FROM Students  
ORDER BY Age DESC;
```

- Retrieves names and ages, sorted **from oldest to youngest**.

4. Why Use SELECT ? (Real-Life Use Cases)

- **Schools** → Generate student lists, attendance reports.
- **Banks** → View customer transactions.
- **Hospitals** → Display patient records.
- **E-commerce** → Show products, prices, and categories.
- **Social Media** → Retrieve posts, likes, and user data.

5. Key Notes to Remember

- `SELECT *` is good for quick checks, but avoid in production (too much unnecessary data).
- Always select only the **columns you need**.
- Use **aliases** (`AS`) to make output readable.
- Combine with functions (`COUNT` , `SUM` , `AVG` , etc.) to get deeper insights (covered in later modules).

6. Classwork (Practice)

👉 Try these queries on **SchoolDB**

1. Show all student **FirstNames** and **LastNames** only.
2. Show all courses with their **CourseCode**.
3. Create a column `FullName` that combines First and Last name.
4. Show all distinct `GradeLevel` values.
5. Show each student's `FirstName` , `Age` , and their `Age in Months` (`Age * 12`).
6. Select all male students and rename the column `Gender` to `Sex` .
7. Display all students but order them by **FirstName alphabetically**.

7. Challenge Question

👉 Write a query to list all students in **Grade 10** and display:

- Their **Full Name** (First + Last)
- Their **Age**
- Their **Grade Level** (renamed as `class`)

Alright 👉 Let's move step by step.

I'll first **solve the classwork from Module 1** (`SELECT`) using our `SchoolDB` . After that, we'll flow into **Module 2: FROM** .



Solutions to Module 1 Classwork

1. Show all student **FirstNames** and **LastNames** only.

```
SELECT FirstName, LastName  
FROM Students;
```

2. Show all courses with their **CourseCode**.

```
SELECT CourseName, CourseCode  
FROM Courses;
```

3. Create a column **FullName** that combines **First** and **Last** name.

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName  
FROM Students;
```

4. Show all distinct **GradeLevel** values.

```
SELECT DISTINCT GradeLevel  
FROM Students;
```

5. Show each student's **FirstName**, **Age**, and their **Age in Months (Age * 12)**.

```
SELECT FirstName, Age, (Age * 12) AS AgeInMonths  
FROM Students;
```

6. Select all male students and rename the column Gender to Sex .

```
SELECT FirstName, LastName, Gender AS Sex
FROM Students
WHERE Gender = 'Male';
```

7. Display all students but order them by FirstName alphabetically.

```
SELECT FirstName, LastName, Age, GradeLevel
FROM Students
ORDER BY FirstName ASC;
```



Challenge Question Solution

👉 List all students in **Grade 10** and display:

- Full Name (First + Last)
- Age
- Grade Level (renamed as Class)

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName,
       Age,
       GradeLevel AS Class
FROM Students
WHERE GradeLevel = 'Grade 10';
```

✅ With this, we've fully wrapped up **Module 1: SELECT** .